

Assignment 4

Shapnil Das

June 10, 2026

1 Question:

In macroeconomics, we often debate whether a series is a Random Walk or just Very Persistent. Simulate two variables for $T = 150$ observations:

1. Variable A (The Unit Root): A random walk with a drift of 0.2.
2. Variable B : A stationary AR(1) process with $\rho = 0.96$ and a deterministic trend of $0.2t$.

Run an Augmented Dickey-Fuller (ADF) test on both. Did the ADF test correctly identify Variable B as stationary? Use Monte-Carlo simulations to show why there is this debate.

2 Solution:

2.1 ADF Test:

We were provided instructions to simulate a Random walk with a drift process which can show **Difference Stationarity** and a stationary AR(1) process with a deterministic trend which shows **Trend Stationarity**.

- **The Unit Root: A random walk with a drift of 0.2**

$$y_t = 0.2 + y_{t-1} + \varepsilon_t \quad (1)$$

where ε_t is a white noise error term.

We can write this equation as:

$$y_t = 0.2t + \sum_{i=1}^t \varepsilon_i \quad (2)$$

- A stationary AR(1) process with $\rho = 0.96$ and a deterministic trend of $0.2t$

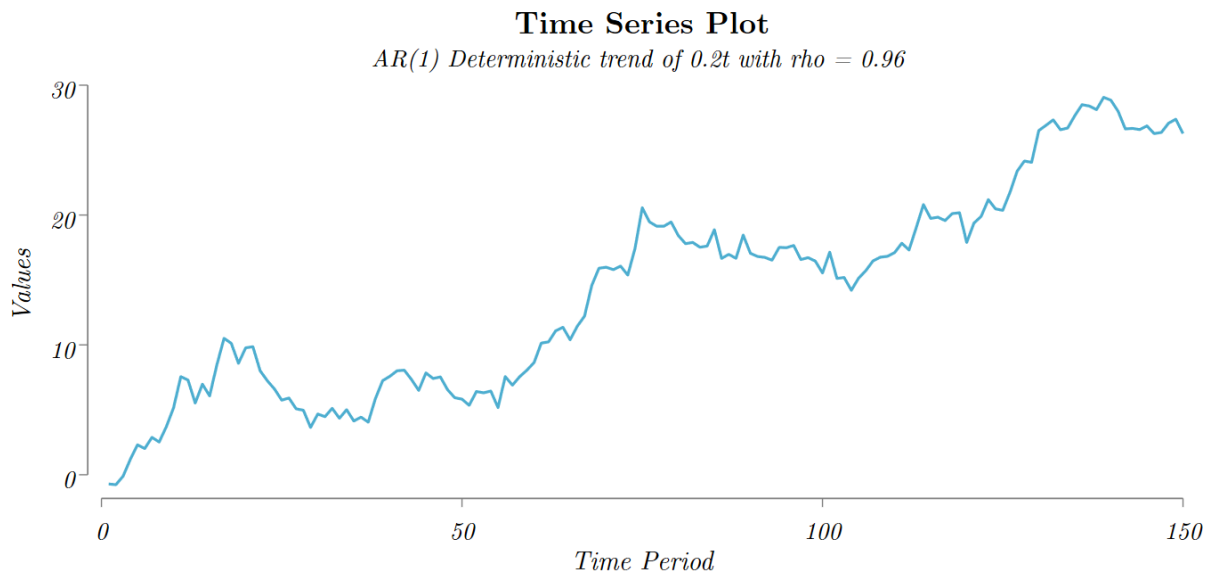
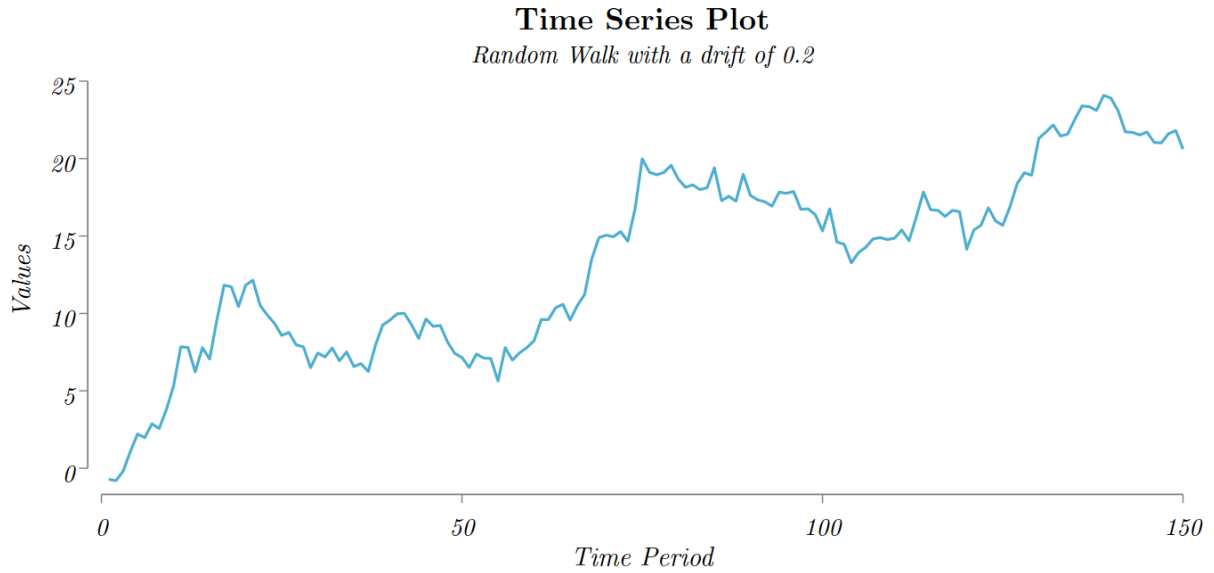
$$x_t = 0.2t + \eta_t$$

where

$$\eta_t = 0.96\eta_{t-1} + \varepsilon_t$$

and ε_t is a white noise error term.

Theoretically, it can be hard to distinguish between a random walk with a drift and a Trend Stationary process with a deterministic trend with visualizations.



We can see that both the series have an upward trend and it is hard to visually identify which one is a random walk and which one is a stationary process with a deterministic trend. So we need to perform an Augmented Dickey-Fuller (ADF) test to identify which one is a random walk and which one is a stationary process. First we have to check for the number of lags in each variable. Using the vector autoregression varsoc command, we can identify the optimal number of lags for each variable. The optimal lag is 1. So we run an ADF test with lags(1).

Table 1: ADF Test of Random walk with drift

	Test	Dicky Fuller Critical Values		
	Statistics	1%	5%	10%
$Z(t)$	-2.596	-4.024	-3.444	-3.144

MacKinnon approximate p-value for $Z(t) = 0.2820$.

The ADF test correctly identifies Variable A as a random walk with a drift and fails to reject the null hypothesis of a unit root. The p-value is 0.2820 which is greater than 0.05, which means we fail to reject the null hypothesis of a unit root.

Table 2: ADF Test of AR (1) with $\rho = 0.96$ deterministic trend variable

	Test	Dicky Fuller Critical Values		
	Statistics	1%	5%	10%
$Z(t)$	-2.276	-4.024	-3.444	-3.144

MacKinnon approximate p-value for $Z(t) = 0.4475$.

Even though variable B is a stationary process, the ADF test fails to identify Variable B as a stationary process with a deterministic trend and fails to reject the null hypothesis of a unit root with a p-value of 0.4475.

2.2 Monte-Carlo Simulation:

Earlier we have seen the ADF test fails to identify Variable B as a stationary process. To investigate why this happens we perform a Monte-Carlo Simulation to do a Type-2 Error Power test to check how many times the ADF test do not reject the false null hypothesis of a unit root when the alternative hypothesis is true and the series is actually stationary.

We use Stata's Program command to simulate 1000 estimation of ADF test statistics of 150 sample for variable A and Variable B. Then we generate a dummy variable which calculates

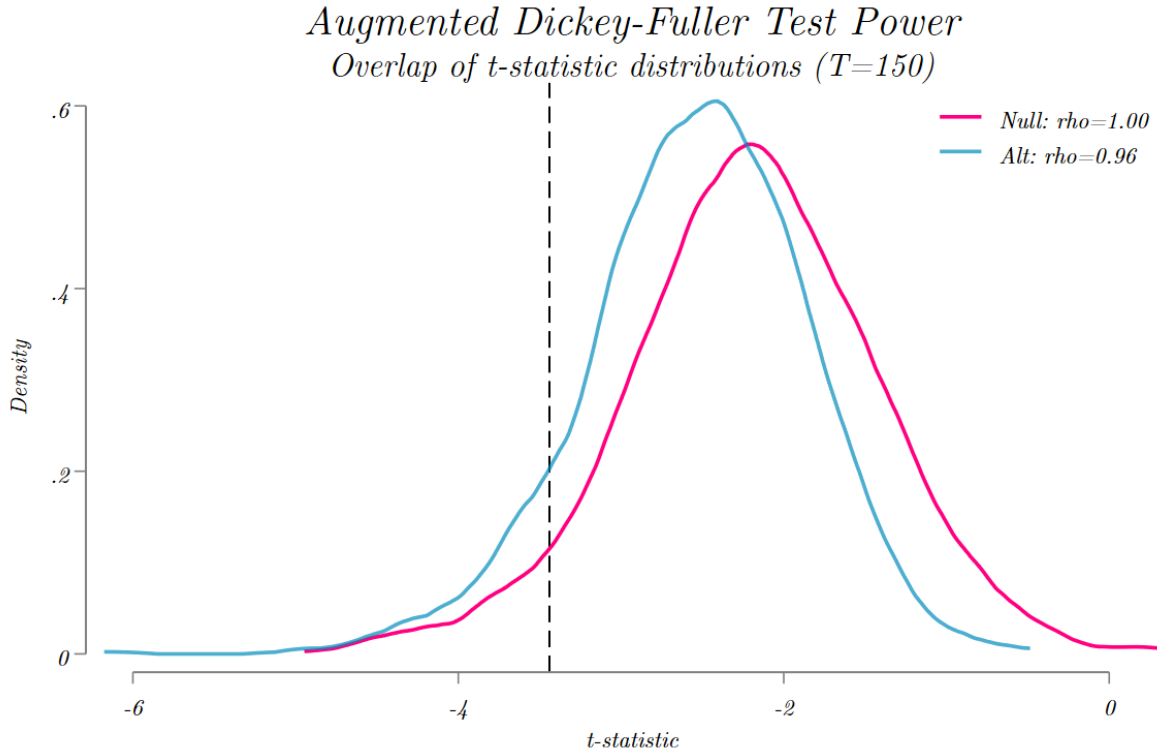
how many times the ADF test statistics is less than -3.44 . Here the mean result is presented:

Table 3: SIMULATION RESULTS: Percentage of Rejection

Data Generating Process	Observations	Mean	Std. Dev.
Random Walk with a drift	1000	0.056	0.230
Trend Stationary ($\rho = 0.96$)	1000	0.099	0.298

For the Random walk process we see that ADF test correctly identifies the unit root process almost 95% times which aligns with the 95% confidence interval at rejection rate 5%. The Type-1 error of false rejection is only 5.6%.

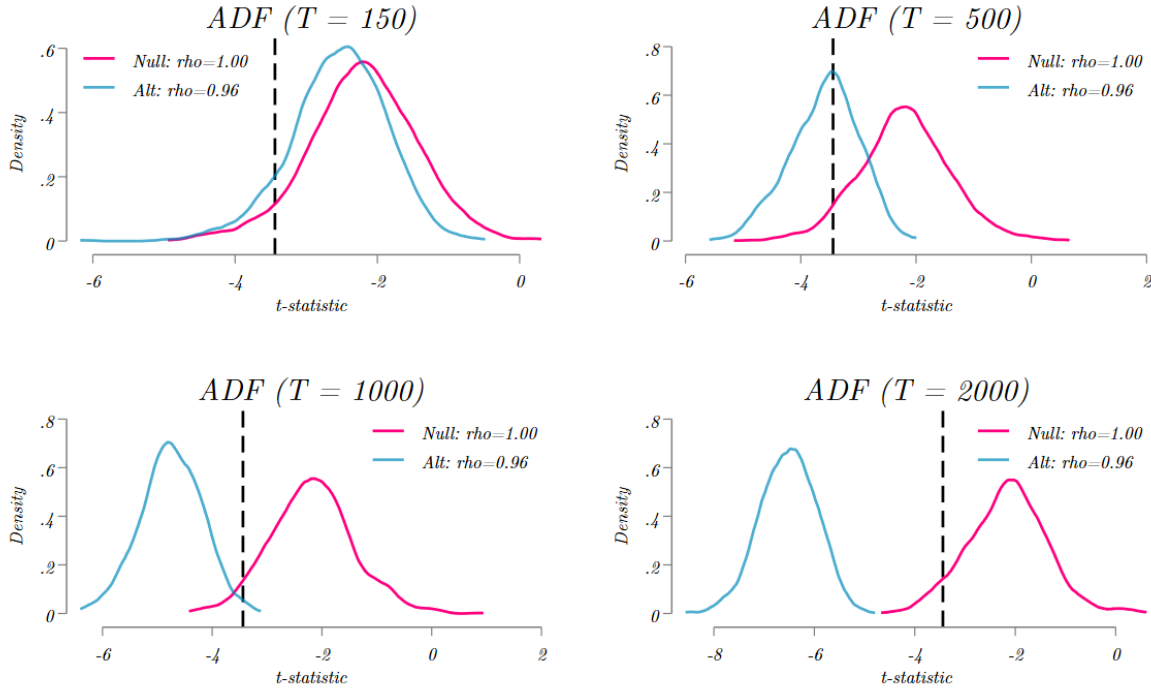
For the stationary deterministic trend process, considering the rejection rate at 5%, almost 95% of the time the ADF test should reject the null hypothesis of a unit root. But we see the complete opposite result. The ADF test rejects the null hypothesis of unit root only 9.9% times with a Type-2 error of almost 90% resulting in a low statistical power test of Augmented Dickey-Fuller test.



From the density plot,

the pink curve shows the unit root process with 95% confidence interval at t-stat -3.44 at the vertical dotted black line. The blue curve shows the stationary process with $\rho = 0.96$. Theoretically, the blue curve should fall completely to the left of the rejection area. Any area right to the dotted black line represents the Type-2 error for blue curve. But the simulations exposes the huge Type-2 error of the blue curve and the unreliability of the ADF test to identify a stationary process when the process is near unit root process.

Law of Large Number



The law of large numbers, a statistical property that says if we increase the number of samples, the estimated or simulated distribution will follow the true distribution of the population mean, can help us to validate the low statistical power of ADF test. We have taken 4 different sample sizes of 150, 500, 1000, 2000 and simulated each sample 1000 times to check the Law of large numbers property. We can see from the stacked diagram that as we increase the sample from 150 to 2000 sequentially, the simulated distribution moves to the left of the true distribution resulting in lower Type-2 error and higher power of the ADF test. So we can conclude that the ADF test has a low power to identify a stationary process when the process is near unit root process and we need to increase the sample size to increase the power of the ADF test. This theory also validates the Monte-Carlo simulation results that we have obtained in this assignment as we have simulated each sample 1000 times, with increased simulated sample the Monte-Carlo simulation can follow the true distribution of

the population mean. But in macroeconomics, we often have limited number of small sample that's why it's not easy to identify a stationary process with a deterministic trend when the process is near unit root process and this is why there is a debate in macroeconomics whether a series is a Random Walk or just Very Persistent.

```

1  cd "F:\ECON 408\Practice do
   file\Assignments\Time-Series-Assignment\Assugment-45\Do
   file"
2
3  clear all
4  set more off
5  set seed 10
6
7  set obs 150
8  gen t = _n
9  tsset t
10
11 set scheme white_tableau
12 graph set window fontface "CMU Serif"
13 graph set eps fontface "CMU Serif"
14 graph set svg fontface "CMU Serif"
15
16 * Step-1: Augmented Dicky Fuller Test of 2 variables
17 *-----
   -----
18
19 * Random Walk with a drift of 0.2
20
21 gen double e = rnormal()
22 gen y_drift = sum(0.2 + e)
23
24 tsline y_drift, ///
25         lcolor(ebblue%70) lwidth(medthick) ///
26         title("{bf:Time Series Plot}", size(large)) ///
27         subtitle("Random Walk with a drift of 0.2") ///
28         xtitle("Time Period", size(medlarge)) ///
29         ytitle("Values", size(medlarge)) ///
30         ylabel(, labsize(medium) nogrid) ///
31         xlabel(, labsize(medium) nogrid) ///
32         graphregion(color(white)) xsize(8) ysize(4) ///
33         name(tsplot_hw4_1, replace)
34
35 graph export tsplot_hw4_1.png, name(tsplot_hw4_1) replace
36
37 varsoc y_drift, maxlag(12)
38 dfuller y_drift, trend lags(1)
39

```

```

40
41 * AR(1), rho = 0.96, deterministic trend of 0.2t.
42
43 gen double u = e in 1
44 replace u = 0.96*L.u + e in 2/L
45 gen double y_trend = 0.2*t + u
46
47
48 tsline y_trend, ///
49         lcolor(ebblue%70) lwidth(medthick) ///
50         title("{bf:Time Series Plot}", size(large)) ///
51         subtitle("AR(1) Deterministic trend of 0.2t with
rho = 0.96 ") ///
52         xtitle("Time Period", size(medlarge)) ///
53         ytitle("Values", size(medlarge)) ///
54         ylabel(, labsize(medium) nogrid) ///
55         xlabel(, labsize(medium) nogrid) ///
56         graphregion(color(white)) xsize(8) ysize(4) ///
57         name(tsplot_hw4_2, replace)
58
59 graph export tsplot_hw4_2.png, name(tsplot_hw4_2) replace
60
61
62 varsoc y_trend, maxlag(12)
63 dfuller y_trend, trend lags(1)
64
65
66 *Step-2: Generating and Simulating Mone-Carlo Simulation
67 *-----
68
69
70 capture program drop sim_power
71 program sim_power, rclass
72
73     syntax [, obs(integer 150)]
74
75     quietly {
76
77         drop _all
78         set obs `obs'
79         gen t = _n

```



```

80         tsset t
81
82         * Random Walk with a drift of 0.2
83
84         gen double e = rnormal()
85         gen y_drift = sum(0.2 + e)
86         dfuller y_drift, trend lags(1)
87
88         return scalar t_drift = r(Zt)
89
90         * AR(1), rho = 0.96, deterministic trend of
91         0.2t.
92
93         gen double u = e in 1
94         replace u = 0.96*L.u + e in 2/L
95         gen double y_trend = 0.2*t + u
96         dfuller y_trend, trend lags(1)
97
98         return scalar t_trend = r(Zt)
99     }
100 end
101
102 simulate t_drift = r(t_drift) t_trend = r(t_trend), reps(
103 1000) seed(10): sim_power, obs(150)
104
105 * STEP 3: Analyze and Visualize the T-statistic
106 *-----
107 --
108
109 gen reject_drift = (t_drift < -3.44)
110 gen reject_trend = (t_trend < -3.44)
111
112 display "--- SIMULATION RESULTS (PERCENTAGE OF
113 REJECTIONS) ---"
114
115 sum reject_drift reject_trend
116
117 twoway ///
118     (kdensity t_drift, lcolor(pink) lwidth(medthick)) ///

```

```
118      (kdensity t_trend, lcolor(ebblue%70) lwidth(medthick
119      )), ///
120      title("Augmented Dickey-Fuller Test Power") ///
121      subtitle("Overlap of t-statistic distributions
122      (T=150)") ///
123      xtitle("t-statistic") ///
124      ytitle("Density") ///
125      xlabel(,nogrid) ylabel(,nogrid) ///
126      xline(-3.44, lcolor(black) lpattern(dash) lwidth(
127      medthick)) ///
128      legend(order(1 "Null: rho=1.00" 2 "Alt: rho=0.96") ///
129      col(1) pos(2) ring(0)) name(hw4_kdensity,
130      replace)
131
132      graph export hw4_kdensity.png, name(hw4_kdensity) replace
133
```